

实验4：中断处理与时钟管理

概述

项目	说明
实验编号	Lab 4 of 8
框架目录	whu-oslab-skeleton/
本实验涉及文件	kernel/boot/start.c kernel/trap/kernelvec.S kernel/trap/trap.c
验收标准（基础）	make run 后终端定期打印 Tick! 时钟心跳信息，系统不崩溃。
验收标准（进阶）	在 QEMU 终端键入任意字符，能立即回显到屏幕（PLIC + UART 外部中断）。

框架文件速查

本实验任务	对应文件	TODO标注
任务1： 设置 stvec 陷阱向量	kernel/trap/trap.c	TODO [Lab4-任务1]
任务2： 保存/恢复所有寄存器	kernel/trap/kernelvec.S	TODO [Lab4-任务2-步骤1,2]
任务3： 时钟中断分发处理	kernel/trap/trap.c	TODO [Lab4-任务3-步骤1]
任务4： 初始化 M-Mode 定时器	kernel/boot/start.c	TODO [Lab4-任务4-步骤1~4]
进阶：PLIC + 外部中断	kernel/trap/trap.c	case 9: 分支

第一节：从顺序执行到事件驱动

在前三个实验中，内核代码”顺序执行”，由 `start_main()` 挨个调用函数后进入死循环。这种模型无法处理外部世界的异步事件：

- 硬件定时器到了，内核不知道
- 键盘有人按键了，内核不知道

- 磁盘读取完成了，内核不知道

中断（Interrupt）和异常（Exception）机制解决了这个问题：硬件事件发生时，CPU 强行中止当前执行，保存现场，跳转到预设的处理程序，完毕后恢复如初继续执行。这个机制是实现时间片调度、I/O 驱动和系统调用的共同基石。

第二节：RISC-V 中断体系结构

2.1 陷阱（Trap）的两大来源

RISC-V 将中断和异常统称为**陷阱（Trap）**：

类型	触发机制	典型例子
中断（Interrupt）	异步，由 CPU 外部硬件信号触发	时钟中断、UART 键盘输入、磁盘完成
异常（Exception）	同步，由当前指令本身触发	非法指令、页错误、系统调用 <code>ecall</code>

核心区分：中断是“外面有人敲门”（CPU 可在任意指令间隙响应），异常是“执行当前指令时自己出了问题”（立即响应，无法延迟）。

2.2 RISC-V 的三个特权级别

1

M-Mode (Machine Mode)

← CPU 上电后首先进入，权限最高

2

↓ mret 降权

3

S-Mode (Supervisor Mode)

← 操作系统内核运行在此

4

↓ sret 降权

5

U-Mode (User Mode)

← 用户程序运行在此，权限最低

本实验的关键点：CLINT（Core Local Interruptor，时钟中断产生器）只能在 M-Mode 下配置。系统必须先在 M-Mode 完成时钟初始化，再通过特殊机制将时钟信号”传递”给 S-Mode 处理。这正是 `start.c` 中 `timerinit()` 的职责。

2.3 中断委托机制（mideleg / medeleg）

若没有委托机制，S-Mode 下发生的 UART 中断需先陷入 M-Mode 再转发回 S-Mode，造成额外的两次特权级切换，效率极低。

RISC-V 提供两个委托寄存器来解决这个问题：

- `medeleg`（Machine Exception Delegation）：将指定异常直接委托给 S-Mode 处理
- `mideleg`（Machine Interrupt Delegation）：将指定中断直接委托给 S-Mode 处理

请阅读 `kernel/boot/start.c` 中的 `start()` 函数，找到委托相关的寄存器写入配置，理解哪些中断类型被委托给了 S-Mode。

思考：时钟中断在 M-Mode 产生，`mideleg` 将其委托给了 S-Mode.....但 M-Mode 的 `timerverc` 处理程序仍然会被调用，它是怎么把信号”移交”给 S-Mode 的？请结合 2.6 节

的路径图思考。

2.4 PMP: M-Mode 对 S-Mode 的内存访问授权

PMP (Physical Memory Protection, 物理内存保护) 是 RISC-V 的一项安全机制, 允许 M-Mode 控制低特权级 (S-Mode、U-Mode) 能访问哪些物理内存范围。其核心规则是:

若无任何匹配的 PMP 条目, S-Mode 的所有内存访问 (取指、读、写) 默认被拒绝。

这意味着: 在 `mret` 降权到 S-Mode 之前, 若不配置 PMP, CPU 执行第一条 S-Mode 指令时就会触发 PMP 访问故障, 系统立刻崩溃, 甚至来不及打印任何信息。

本框架在 `start()` 函数中已预置了最简单的“全通行证”配置:

```
1 // pmpaddr0: TOR 模式下覆盖物理地址 [0, pmpaddr0*4) ≈ 全部物理内存
2 asm volatile("csrw pmpaddr0, %0" : : "r"(0x3fffffffffffffff));
3 // pmpcfg0 = 0x0F = R(1) | W(1) | X(1) | A=TOR(01)
4 // 效果: 赋予 S-Mode 对全部物理内存的读/写/执行权限
5 asm volatile("csrw pmpcfg0, %0" : : "r"(0xf));
```

`pmpcfg0 = 0x0F` 的各位含义:

位	字段	值	含义
bit 0	R	1	S-Mode 可读
bit 1	W	1	S-Mode 可写
bit 2	X	1	S-Mode 可执行
bit[4:3]	A	01	TOR 模式 (Top Of Range, 从 0 到 <code>pmpaddr0</code>)

这两条语句已由框架提供, 无需修改。理解它们的存在原因即可——这是打开 S-Mode“运行权限”的必经一步。

2.5 陷阱触发时硬件自动完成的 4 件事 (必须理解!)

当陷阱在 S-Mode 下触发时, RISC-V 硬件自动且不可打断地执行以下 4 步, 无需任何软件干预:

- ① 保存返回地址: `sepc` ← 当前 PC (记录被打断时的指令地址, 以便后续恢复执行)
 - ② 记录原因代码: `scause` ← 陷阱原因 (bit63=1 为中断, bit63=0 为异常; 低位为具体类型)
 - ③ 记录附加信息: `stval` ← 额外信息 (如缺页故障地址、非法指令编码等)
 - ④ 关闭全局中断: `sstatus.SIE` ← 0 (防止陷阱处理过程中被再次打断)
- 然后 `PC` ← `stvec` (跳转到你设置的陷阱处理入口地址)

注意: 硬件只保存了几个 CSR 寄存器, 全部 32 个通用寄存器 (`ra`, `sp`, `a0` - `a7`, `s0` - `s11` 等) 必须由软件手动保存! 这正是任务 2 的工作。

软件接下来必须完成 (你的任务):

- ⑤ 汇编 (`kernelvec.S`): 将 32 个通用寄存器压栈保存
- ⑥ C 函数 (`trap.c`): 读 `scause`, 分析中断/异常类型, 分发给具体处理函数

- 3 ⑦ 汇编 (kernelvec.S) : 从栈上恢复 32 个通用寄存器
- 4 ⑧ sret 指令返回 (硬件自动: PC ← sepc, sstatus.SIE 恢复, 返回原特权级)

2.6 scause 寄存器编码 (本实验需要)

scause 寄存器的结构: 最高位 (bit 63) 为 1 时表示异步中断, 为 0 时表示同步异常; 低位为具体编号。

scause 值	含义	本实验涉及
0x8000000000000001	S-Mode 软件中断	✔ 任务3: 时钟心跳
0x8000000000000005	S-Mode 时钟中断	框架已处理
0x8000000000000009	S-Mode 外部中断	✔ 进阶: PLIC+UART
0x0000000000000008	U-Mode ecall	Lab6 实现

参考资料: 请阅读 kernel/include/riscv.h 中的 r_scause()、r_sstatus() 等函数定义, 了解各 CSR 的读写接口。

2.7 时钟中断的完整信号传递路径

理解以下路径是完成任务4的关键:

```
1 CLINT 硬件: mtime ≥ mtimecmp
2   → 触发 M-Mode 时钟中断
3   → CPU 跳转到 mtvec 指向的 timervec (M-Mode 汇编, 框架已提供)
4   → timervec 执行两件事:
5       1. 将 mtimecmp += interval (安排下一次中断时间)
6       2. 向 sip.SSIP 写入 1 (向 S-Mode 注入一个"软件中断"信号)
7   → mret 返回 (M-Mode 结束)
8   → S-Mode 收到信号: scause = 0x8000000000000001 (软件中断)
9   → CPU 跳转到 stvec 指向的 sys_trap_vector (你的汇编入口)
10  → sys_trap_handler() 中 case 1: 处理时钟事件
```

第三节: 任务实施指南

阅读要求: 完成每个任务前, 请先阅读对应框架文件的完整注释, 理解每个函数的目标和约束。任务指南只提供方向和关键知识点, 不给出具体代码。

任务1 / kernel/trap/trap.c 一向 CPU 注册陷阱入口地址

目标: 在 trapinithart() 函数中告诉 CPU, 当 S-Mode 下发生中断或异常时, 跳转到哪个地址处理。

你需要了解的知识：

- `stvec` (S-Mode Trap Vector Register) 是 RISC-V 的 CSR 之一，存放 S-Mode 陷阱处理入口地址。
- 该寄存器的写接口为 `w_stvec(uint64 x)`，已在 `kernel/include/riscv.h` 中定义。
- 汇编文件 `kernelvec.S` 中定义了入口标签 `sys_trap_vector`，在 C 中已用 `extern char sys_trap_vector[]` 声明。

关键问题：如何将一个汇编标签的地址转换成 `uint64` 类型传给 `w_stvec`？

验证方法：在 GDB 中断点于 `trapinithart`，执行后检查 `csrr a0, stvec` 的值是否与 `sys_trap_vector` 标签地址一致。

任务2 / `kernel/trap/kernelvec.S` — 保护和恢复完整的 CPU 状态

目标：在调用 C 函数 `sys_trap_handler()` 前，将全部 32 个 RISC-V 通用寄存器保存到当前内核栈上；返回后再完整恢复。

你需要了解的知识：

框架已经为你完成了：

- 栈空间分配：`addi sp, sp, -256` (256 = 32 个寄存器 × 8 字节)
- `ra`，`gp`，`tp`，`t0` - `t2` 的保存示例（以及对应的恢复代码）
- `sp` 自身的特殊处理（已通过 `addi t0, sp, 256` 还原后存入 `8(sp)`）

RISC-V 的 32 个通用寄存器（按 ABI 名称）：

```
1 x0/zero x1/ra x2/sp x3/gp x4/tp
2 x5-x7/t0-t2 x8-x9/s0-s1
3 x10-x17/a0-a7 x18-x27/s2-s11 x28-x31/t3-t6
```

步骤1 规则：每个 `sd <寄存器>, <偏移>(sp)` 的偏移量从 0 开始，每次递增 8。参照框架已给出的示例（`ra` 在偏移 0，`sp` 在偏移 8，`gp` 在偏移 16...），按照寄存器编号顺序（x1, x2 ... x31）依次补全所有缺失的 `sd` 指令。

步骤2 规则：恢复时使用 `ld` 指令，顺序必须与 `sd` 完全对称镜像。**注意：**`sp` 必须最后恢复，否则其后的 `ld` 指令使用已改变的基地址会出错。

深度思考题：框架代码中，`tp` (thread pointer) 寄存器保存了，但 xv6 原版在恢复时有时故意跳过 `tp`。这是为什么？`tp` 在本内核里被用来存什么重要信息？（提示：查阅 `kernel/include/riscv.h` 中与 `tp` 寄存器相关的操作）

任务3 / `kernel/trap/trap.c` — 识别并处理时钟中断信号

目标：在 `sys_trap_handler()` 的 `case 1:` 分支（S-Mode 软件中断）中，正确地响应由 M-Mode `timervec` 注入的时钟信号，并打印心跳信息。

你需要完成的两件事：

1. **清除中断待处理标志：**`timervec` 通过向 `sip` 寄存器的 SSIP 位 (bit 1) 写 1 来触发软件中断。处理完后，你必须将 SSIP 清零，否则这个中断会无限重复触发形成死循环。
 - 读写接口：`r_sip()` / `w_sip(x)` (见 `riscv.h`)
 - SSIP 对应 `sip` 寄存器的 bit 1

2. 统计 ticks 并定期打印：维护一个时钟中断计数器 `ticks`，每当计数到一定倍数时（例如每 10 次）打印一次 `Tick!`，避免每次触发都刷屏。

关键问题（必须回答）：如果改为每次时钟中断都打印 `Tick!`，以 `interval=1000000` 的频率（约10次/秒），你估计屏幕输出速率会是多少？这对系统调试的影响是什么？

任务4 / `kernel/boot/start.c` — 配置 M-Mode 硬件定时器

目标：在 `timerinit()` 中完成 CLINT 定时器的4步初始化，使其能定期产生时钟中断并触发整个中断链路。

参考资料（完成任务前请逐一阅读）：

要了解的内容	查阅位置
<code>CLINT_MTIME</code> <code>CLINT_MTIMECMP</code> 宏的物理地址定义	<code>kernel/include/memlayout.h</code>
<code>w_mtvec()</code> <code>r_mie()</code> <code>w_mie()</code> <code>r_mstatus()</code> <code>w_mstatus()</code> 的定义	<code>kernel/include/riscv.h</code>
<code>MIE_MTIE</code> <code>MSTATUS_MIE</code> 等位掩码常量	<code>kernel/include/riscv.h</code>
<code>timervec</code> 汇编入口的作用（为什么需要它）	<code>kernel/boot/start.c</code> 顶部注释
<code>timer_scratch</code> 和 <code>mscratch</code> 寄存器的关系	<code>kernel/boot/start.c</code> 代码注释

4步任务说明：

- 步骤1 — 安排第一次中断时刻：向 CLINT 的硬件寄存器 `MTIMECMP[hartid]` 写入“当前时间 + 间隔”，使得定时器在间隔时间后首次触发。请思考：如何通过 `memlayout.h` 中的宏计算出具体的内存映射地址？
- 步骤2 — 注册 M-Mode 陷阱入口：将 `timervec`（M-Mode 专用时钟中断汇编处理程序）的地址写入 `mtvec` CSR。思考：M-Mode 下的 CSR 写函数是 `w_mtvec` 还是 `w_stvec`？为什么不同？
- 步骤3 — 使能 M-Mode 时钟中断：`mie`（Machine Interrupt Enable）寄存器的 MTIE 位控制 M-Mode 时钟中断的使能开关。请使用“读-改-写”模式设置该位，不要覆盖其他位。
- 步骤4 — 开启 M-Mode 全局中断：`mstatus.MIE` 是 M-Mode 下的全局中断总开关，不打开则所有 M-Mode 中断都被屏蔽。同样使用“读-改-写”模式。

整体思考题： 完成 4 步后，请描述一次完整的时钟中断触发路径：从”硬件 CLINT 到 M-Mode timervec，再到 S-Mode sys_trap_handler，最后到你的 `Tick!` 打印”，途经哪些 CSR 被读写，哪些函数或汇编被执行？

第四节：运行与基础验收

```
1 make run
```

预期输出（每 0.1 秒左右刷新一次）：

```
1 Paging enabled! Interrupts on.
2 Tick! (10)
3 Tick! (20)
4 ...
```

按 `Ctrl+A` 然后 `X` 退出 QEMU。

第五节：进阶任务——PLIC 与 UART 外部中断

5.1 为什么需要 PLIC?

在仅有时钟中断的情况下，QEMU 的键盘输入会被忽略。真实操作系统需要能”监听”外部设备的请求。PLIC (Platform-Level Interrupt Controller, 平台级中断控制器) 是 RISC-V 的”外部中断集线器”，负责汇集所有外设的中断信号，统一分配给 CPU 核心处理。

PLIC 的中断链路：

```
1 键盘按键 → UART 硬件产生 IRQ 信号
2           → PLIC 接收 IRQ (需要事先配置优先级和使能)
3           → PLIC 向 CPU 发出外部中断 (scause = 0x8000000000000009)
4           → 内核查询 PLIC : 是哪个设备 (Claim)
5           → 处理该设备的请求
6           → 通知 PLIC 处理完毕 (Complete)
```

5.2 PLIC 配置（需要实现）

PLIC 的所有寄存器均通过内存映射访问，地址常量已在 `kernel/include/memlayout.h` 中定义（以 `PLIC_` 为前缀）。请自行阅读该文件，理解以下寄存器的含义：

- `PLIC_PRIORITY` : 各设备的中断优先级配置
- `PLIC_SENABLE(hartid)` : S-Mode 下该核心对各设备中断的使能配置
- `PLIC_SPRIORITY(hartid)` : S-Mode 下该核心的优先级阈值
- `PLIC_SCLAIM(hartid)` : S-Mode 下查询 (Claim) /完成 (Complete) 中断的寄存器

UART0 的 IRQ 编号： `UART0_IRQ` （已在 `memlayout.h` 中定义）

你需要实现：

1. 在初始化阶段（可以在 `trap.c` 或新建 `plic.c` ）配置 PLIC，使其能将 UART0 的 IRQ 中断发送给当前核心（S-Mode）
2. 在 `sys_trap_handler` 的 `case 9:` 分支中，实现外部中断处理逻辑：
 - Claim：向 PLIC 查询是哪个设备
 - 处理 UART0 IRQ：从 UART 的 RHR 寄存器（偏移 0）读取字节并回显
 - Complete：通知 PLIC 处理完毕

关键问题：为什么处理完设备后必须向 PLIC_SCLAIM 写回 IRQ 编号（Complete 操作）？若省略，下一次按键还能触发中断吗？

5.3 进阶验收

`make run` 后在 QEMU 终端中随意输入字符，若能立即回显则说明 PLIC 与外部中断配置正确。

第六节：调试与故障排除

症状	优先排查
系统启动后无 <code>Tick!</code>	① <code>trapinithart()</code> 是否在 <code>main.c</code> 中被调用？ ② <code>intr_on()</code> 是否开启？ ③ <code>timerinit()</code> 4步是否全部完成？
首次 <code>Tick!</code> 后系统崩溃	<code>kernelvec.S</code> 寄存器保存/恢复是否完全对称？缺一个 <code>ld</code> 都会导致返回地址神秘损坏
<code>Tick!</code> 出现后陷入死循环打印	<code>case 1:</code> 中是否清除了 <code>sip.SSIP</code> 位？
无 <code>Tick!</code> 且系统卡死不输出	用 GDB 断点于 <code>trapinithart</code> 检查 <code>stvec</code> 的值是否正确
键盘无回显（进阶）	① PLIC 优先级是否设为 <code>> 0</code>？ ② 该核心的使能寄存器是否设置？ ③ 阈值是否设为 <code>0</code>？

第七节：思考题

1. RISC-V 的 `sret` 指令执行时，硬件会自动完成哪些动作？与软件的 `w_sepc()` / `w_sstatus()` 是什么关系？
2. 如果将 `mideleg` 全部清零（不委托任何中断），操作系统内核仍在 S-Mode 运行，此时 UART 按键中断到来，整个系统会如何响应？

3. 完整路径描述（核心考察）：从“用户在 QEMU 键盘上按下一个字符”到“该字符回显在屏幕上”，完整描述涉及的硬件动作、寄存器变化、特权级切换和软件函数调用链（若完成进阶任务）。
 4. 为什么内核在调用 `sys_trap_handler` 前要关闭中断（`sstatus.SIE`）？如果不关，在处理一个 UART 中断期间又来了一个时钟中断会怎样？
-

第八节：实验完成后的全貌

Makefile SRCS: `entry.S + start.c + uart.c + main.c + console.c + kalloc.c + vm.c + kernelvec.S + trap.c`

`start_main()` 应有的调用顺序：

```
1 void start_main() {
2     kinit(); kmininit(); kminithart(); // Lab3 : 内存与分页
3     trapinithart(); // Lab4 : 注册陷阱向量
4     // (进阶) plicinit(); // Lab4 进阶 : 配置 PLIC
5     intr_on(); // Lab4 : 开启 S-Mode 全局中断
6     clear_screen();
7     printf("Paging enabled! Interrupts on.\n");
8     while(1);
9 }
```

Lab5 预告：下一个实验将在时钟中断 `case 1:` 里加入 `yield()` 调用，并实现进程控制块（PCB）的分配和汇编级上下文切换（`swtch.S`），操作系统将第一次具备真正的多任务能力。